



REAL-TIME SWITCHED-MODE POWER CONVERTER SIMULATOR

STM32F4 – Discrete-Time Plant + PI Control + PWM + UART CLI

Lappeenranta–Lahti University of Technology LUT

Master Programme in Electrical Transportation Systems

2025

Hesam Vahib (3073128), David Rybæk (3144811), Fatemeh Kholardi (3278363)

Examiner: Professor Tuomo Lindh

ABSTRACT

Lappeenranta–Lahti University of Technology LUT
LUT School of Engineering Sciences
Master Programme in Electrical Transportation Systems

Hesam Vahib, David Rybàk, Fatemeh Kholardi

REAL-TIME SWITCHED-MODE POWER CONVERTER SIMULATOR

STM32F4 – Discrete-Time Plant + PI Control + PWM + UART CLI

Embedded System Programming Project

2025

22 pages, 7 figures, 3 tables and 1 appendix

Examiner: Professor Tuomo Lindh

Keywords: Real-time embedded system, Closed-loop control, PI controller, Pulse Width Modulation (PWM), UART communication

This project implements a real-time switched-mode power converter simulator on an STM32F4-series development board using the STM32 HAL. A discrete-time sixth-order state-space plant is simulated at a fixed sampling rate of 1 kHz, and a PI controller computes a PWM duty command to regulate the normalized output toward a reference. The firmware is interrupt-driven: TIM3 provides the 1 kHz control tick, TIM2 generates PWM and can optionally drive the plant input through PWM timer events. A UART command-line interface enables monitoring and parameter changes, while hardware buttons provide mode switching and gain/reference tuning with debounced press detection. Three operating modes are implemented (IDLE, MOD, CONFIG) and verified using UART status outputs and LED indicators. Two bonus features are demonstrated: a sinusoidal inverter-style reference generator and PWM-event-driven plant stepping using discrete input states. The verification results confirm correct mode behavior, stable closed-loop response, and consistent user interaction.

Table of contents

Abstract

1	Introduction	5
2	Hardware and I/O Configuration.....	6
2.1	Microcontroller and peripherals	6
2.2	LEDs (mode indication).....	7
2.3	Buttons	7
3	Software Architecture.....	8
3.1	Timing model and task execution	9
3.2	PWM generation and interrupt handling.....	9
3.3	UART communication and command-line interface	10
3.4	Data protection and user-interface arbitration.....	10
4	Mathematical Model and Control.....	11
4.1	Discrete-time plant model	11
4.2	Measurement normalization.....	11
4.3	Measurement filtering	12
4.4	PI controller with anti-windup	12
5	Operating Modes	13
5.1	IDLE mode.....	13
5.2	MOD mode (closed-loop modulation).....	13
5.3	CONFIG mode (parameter tuning)	14
6	User Interface	14
6.1	UART CLI commands	14
6.2	Buttons and on-board control.....	15
7	Bonus Features	15
7.1	Bonus 1: inverter sinusoidal reference	15
7.2	Bonus 2: PWM-event-driven plant stepping.....	16
8	Verification and Results	17
8.1	Test 1: Boot and IDLE state.....	17
8.2	Test 2: MOD mode tracking	17
8.3	Test 3: CONFIG mode tuning.....	18
8.3.1	Interface arbitration and data protection verification	19
8.4	Test 4: Bonus 1 sine reference	20
8.5	Test 5: Bonus 2 PWM stepping	20
9	Conclusion.....	21

Appendices

Appendix 1. Key Parameters

Figures

Figure 1: STM32F4 development board and hardware setup used for the real-time converter simulator.

Figure 2. Bonus 1 enabled, the reference signal follows a sinusoidal trajectory mapped to the normalized range.

Figure 3. Bonus 2 enabled. Plant input is driven by PWM events with discrete input states and polarity switching.

Figure 4. LEDs are off and UART status confirms a safe startup state with zero PWM duty.

Figure 5. MOD mode operation. PA6 LED indicates modulation mode and UART output shows closed-loop tracking of the reference.

Figure 6. CONFIG mode. Parameter tuning is enabled, indicated by the PA7 LED and the UART configuration lock flag.

Figure 7: verification of interface arbitration using binary and timed semaphores

Tables

Table 1. Mode LEDs

Table 2. Buttons and functions

Table 3. Key Parameters

1 Introduction

This project implements a real-time switched-mode power converter simulator on an STM32F4-series development board (Nucleo-F411RE) using the STM32 HAL. The objective is to demonstrate real-time embedded control of a discrete-time plant model with pulse-width modulation (PWM) actuation, while providing a robust and user-friendly interface for monitoring and tuning.

A sixth-order discrete-time state-space model is simulated at a fixed sampling rate of 1 kHz. A proportional–integral (PI) controller regulates the normalized plant output toward a user-defined reference by computing a PWM duty command. The firmware is fully interrupt-driven, ensuring deterministic timing and low CPU utilization.

User interaction is provided through two complementary interfaces. A UART command-line interface allows inspection of internal variables and modification of controller parameters, while on-board buttons enable mode switching and incremental tuning. Three operating modes are implemented: IDLE for safe operation, MOD for closed-loop control, and CONFIG for parameter tuning. Two additional bonus features extend the functionality: an inverter-style sinusoidal reference generator and a PWM-event-driven plant input model. Correct operation of all modes and features is verified using UART status outputs and LED indicators.

2 Hardware and I/O Configuration

2.1 Microcontroller and peripherals

The system is implemented on an STM32F4-series development board (Nucleo-F411RE), based on the STM32F411RE microcontroller. The firmware is developed using the STM32 Hardware Abstraction Layer (HAL), providing portable and hardware-independent access to peripherals.

The following on-chip peripherals are used:

- **USART2** is configured at 115200 baud (8N1) to provide a bidirectional UART interface for the command-line interface. GPIO pins PA2 and PA3 are used for transmit and receive, respectively.
- **TIM3** is configured to generate a periodic interrupt at 1 kHz. This interrupt defines the discrete-time sampling period of the control loop and plant simulation, corresponding to a sampling time of $T_s = 1\text{ms}$.
- **TIM2** is configured to generate a PWM signal on Channel 1. The PWM duty cycle is updated by the controller in real time. Additionally, TIM2 update and compare interrupts are used to implement PWM-event-driven plant stepping in the bonus configuration.

Figure 1 shows the STM32F4 development board and the hardware setup used for the experimental validation of the converter simulator.

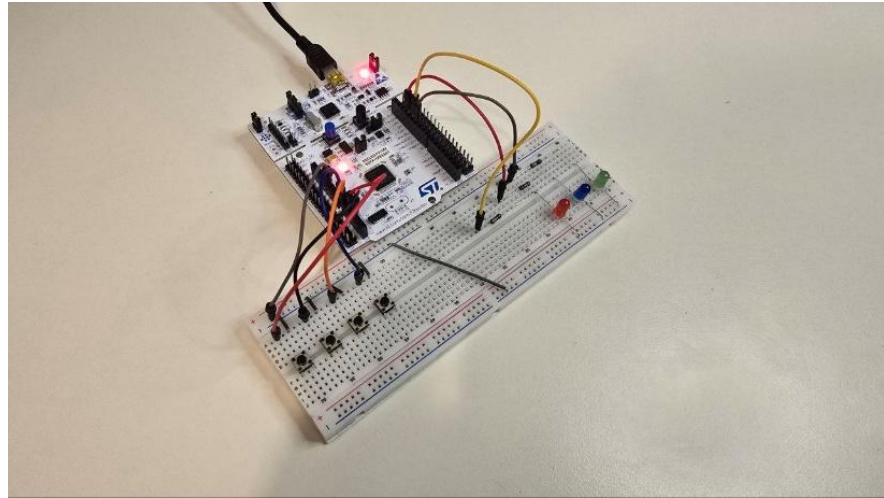


Figure 1. STM32F4 development board and hardware setup used for the real-time converter simulator.

2.2 LEDs (mode indication)

Two on-board LEDs are used to indicate the current operating mode. The LED states are mutually exclusive and directly reflect the active mode of the application, providing immediate visual feedback.

Table 1. Mode LEDs

Mode	LED behavior	GPIO
IDLE	ALL OFF	PA6 = 0, PA7 = 0
MOD	PA6 ON	PA6 = 1, PA7 = 0
CONFIG	PA7 ON	PA6 = 0, PA7 = 1

2.3 Buttons

Four on-board buttons are used for mode control and parameter tuning. All buttons are configured as active-low inputs with internal pull-up resistors. To ensure reliable operation, button presses are debounced in software using an integrator-based method executed at the 1 kHz system tick.

The MODE button cycles the system through the three operating modes: IDLE, MOD, and CONFIG. In CONFIG mode, a SELECT button toggles the active controller parameter between proportional gain K_p and integral gain K_i . Separate INCREASE and DECREASE buttons are used to adjust the selected parameter or, in MOD mode, to modify the reference value. Each button press generates exactly one action, preventing repeated triggering due to mechanical bounce.

Table 2. Buttons and functions

Function	Pin	Type
Mode cycle (IDLE→MOD→CONFIG→IDLE)	PC13	EXTI interrupt
Mode cycle (alternative)	PA0	polled + debounced
Increase	PA1	polled + debounced
Decrease	PB0	polled + debounced
Select parameter (KP/KI toggle in CONFIG)	PA4	polled + debounced

3 Software Architecture

The firmware is designed as a fully interrupt-driven embedded application. After system initialization, the main loop enters a low-power wait-for-interrupt state. All time-critical functionality, including control execution, plant simulation, user input handling, and communication, is performed inside interrupt service routines. This design ensures deterministic timing behavior and minimizes CPU load.

The application logic is separated from hardware initialization. Peripheral configuration is performed in the main program, while all control, simulation, and user interface logic is implemented in the application module. This separation improves readability and simplifies future modifications.

3.1 Timing model and task execution

The real-time behavior of the system is governed by a fixed sampling period of 1 ms. This sampling period is defined by TIM3, which generates a periodic update interrupt at 1 kHz. Each TIM3 interrupt triggers the execution of the main application tick function, where control, plant simulation, and button handling are performed.

The discrete-time sampling parameters are defined as:

$$T_s = 1 \text{ ms} \quad f_s = \frac{1}{T_s} = 1 \text{ kHz} \quad (1)$$

At each sampling instant, the following operations are executed in a fixed and deterministic order:

1. Button states are sampled and debounced, and press events are generated if detected.
2. The current operating mode is evaluated and corresponding actions are executed.
3. The plant output is mapped to a normalized measurement and filtered.
4. The controller computes the new PWM duty command when applicable.
5. The PWM compare register is updated accordingly.

This strictly periodic execution guarantees consistent control performance and numerical stability.

3.2 PWM generation and interrupt handling

TIM2 is configured to generate a PWM signal on Channel 1. The controller output is expressed as a normalized duty command in the range [0,1], which is written to the TIM2 compare register at each control step.

In the standard configuration, the discrete-time plant model is stepped once per sampling period using the averaged PWM duty value. In the bonus configuration, the plant model is instead driven by PWM timer events. Specifically:

- The TIM2 update interrupt corresponds to the start of a PWM period and triggers a plant step with a signed input.
- The TIM2 compare interrupt corresponds to the end of the PWM on-time and triggers a plant step with zero input.

This event-based stepping produces a switching-like excitation of the plant and more closely resembles the behavior of a switched-mode power converter.

3.3 UART communication and command-line interface

UART communication is implemented using USART2 and interrupt-based reception. Incoming characters are received one byte at a time and echoed back to the terminal. Characters are buffered until a line termination character is detected, after which the complete command line is processed.

The command-line interface supports status monitoring, mode selection, controller parameter updates, and activation of bonus features. The status command prints a consistent snapshot of the system state, including operating mode, controller gains, reference value, measurement, PWM duty, plant output, and bonus feature flags.

To avoid inconsistent updates, UART-based parameter modification is disabled in CONFIG mode. This behavior is indicated by a configuration lock flag, ensuring that controller parameters are modified only through the button interface during tuning.

3.4 Data protection and user-interface arbitration

To prevent conflicting parameter updates from different interfaces, two lightweight software semaphores are implemented:

- **Binary semaphore (CONFIG exclusive lock):** entering CONFIG mode *takes* an exclusive lock. While this lock is taken, **all buttons are ignored**, ensuring deterministic tuning ownership and preventing accidental changes. The lock is released when leaving CONFIG mode.
- **Timed semaphore (UART block after button usage):** after any button press that performs an action (MODE/INC/DEC/SELECT), **UART parameter-changing commands are rejected for 5 seconds**, while monitoring commands (e.g., `status`, `help`, and `mode ...`) remain available. This prevents near-simultaneous tuning from UART and buttons.

Critical sections implemented by brief interrupt disable/enable ensure atomic access to shared variables.

4 Mathematical Model and Control

4.1 Discrete-time plant model

The power converter is represented by a discrete-time state-space model of order six. The model is evaluated at the fixed sampling period $T_s = 1\text{ms}$ and is implemented directly in software. The state update equation is given by:

$$\mathbf{x}[k + 1] = \mathbf{A} \mathbf{x}[k] + \mathbf{B} u[k] \quad (2)$$

where $\mathbf{x} \in \mathbb{R}^6$, $\mathbf{A} \in \mathbb{R}^{6 \times 6}$, $\mathbf{B} \in \mathbb{R}^6$, and the output is the sixth state. The matrices $\mathbf{A}$ and $\mathbf{B}$ are defined as constants in the firmware.

where $\mathbf{x}[k] \in \mathbb{R}^6$ is the state vector at time step k , $\mathbf{A} \in \mathbb{R}^{6 \times 6}$ is the discrete-time state transition matrix, $\mathbf{B} \in \mathbb{R}^6$ is the input vector, and $u[k]$ is the plant input.

The plant output is defined as the sixth state:

$$y[k] = x_6[k] \quad (3)$$

The numerical values of $\mathbf{A}$ and $\mathbf{B}$ are fixed and provided as constants in the firmware. To ensure numerical robustness, all state variables and the output are monitored. If any state exceeds a predefined limit or becomes invalid, the plant state is reset to zero to prevent instability.

4.2 Measurement normalization

The controller operates on a normalized measurement signal $\mathbf{m}[k] \in [0,1]$, derived from the plant output. A full-scale output value Y_{FS} is defined for normalization.

In the standard (non-inverter) configuration, the absolute value of the output is used:

$$m[k] = \frac{\min(|y[k]|, Y_{FS})}{Y_{FS}} \quad (4)$$

When inverter-style operation is enabled (Bonus 1), the signed output is mapped symmetrically around the midpoint:

$$m[k] = \text{clamp}(0.5 + 0.5 \frac{y[k]}{Y_{FS}}, 0, 1)$$

where $\text{clamp}(\cdot)$ limits the value to the interval $[0,1]$.

4.3 Measurement filtering

To reduce the influence of noise and fast plant dynamics, the normalized measurement is filtered using a first-order low-pass filter:

$$m_f[k] = m_f[k-1] + \alpha (m[k] - m_f[k-1]) \quad (6)$$

Where $m_f[k]$ is the filtered measurement and α is the filter coefficient. In this implementation, $\alpha = 0.02$, providing a compromise between noise attenuation and responsiveness. The filtered measurement $m_f[k]$ is used as the feedback signal for the controller.

4.4 PI controller with anti-windup

A proportional–integral (PI) controller regulates the normalized plant output. The control error is defined as

$$e[k] = r[k] - m_f[k] \quad (7)$$

The proportional and integral terms are computed as

$$u_p[k] = K_p e[k] \quad (8)$$

$$u_i^{new}[k] = u_i[k-1] + K_i T_s e[k] \quad (9)$$

The unsaturated control command is given by

$$u_{try}[k] = u_p[k] + u_i^{new}[k], \quad (10)$$

To prevent integrator windup, conditional integration is applied: the integrator state is updated only when the control command is within actuator limits or when the error drives a

saturated output back toward the valid range. The final PWM duty command is obtained by saturation:

$$u[k] = \text{clamp}(u_p[k] + u_i[k] \ 0 \ 1), \quad (11)$$

This strategy ensures stable control behavior during saturation and transients.

5 Operating Modes

The application operates in three mutually exclusive modes: IDLE, MOD, and CONFIG. The active mode determines the control behavior, user interaction, and LED indication.

5.1 IDLE mode

Purpose: safe state with no control action.

- PWM duty is set to zero: $u[k] = 0$.
- The plant is stepped with $u = 0$ if Bonus 2 is disabled.
- Measurement is forced to zero.
- LEDs: PA6 OFF, PA7 OFF.

5.2 MOD mode (closed-loop modulation)

Purpose: closed-loop PI tracking. At each 1 ms tick:

- 1) compute measurement mapping $m[k]$ and filter $m_f[k]$,
- 2) set reference $r[k]$ (user reference or Bonus 1 sine reference),
- 3) run PI control to compute duty $u[k]$,
- 4) update TIM2 PWM compare register with $u[k]$,
- 5) step the plant either with averaged input (normal) or via PWM events (Bonus 2).

LEDs: PA6 ON.

5.3 CONFIG mode (parameter tuning)

Purpose: tune controller gains.

- A constant duty $u = 0.10$ is applied.
- SELECT button toggles the active parameter (KP/ KI).
- INC/DEC adjust the selected parameter:
 - $K_p \leftarrow K_p \pm 0.05$
 - $K_i \leftarrow K_i \pm 2.5$ (implemented as $K_i \leftarrow K_i + 50 \cdot (\pm 0.05)$)
- UART configuration lock is indicated in status as `lock = 1`.
- CONFIG exclusive lock is taken: UART-owned CONFIG blocks buttons; button-entered CONFIG allows tuning.
- UART parameter changes are restricted: only `status/help/mode` are accepted while locked/blocked; gain/reference changes via UART are rejected.

LEDs: PA7 ON.

6 User Interface

6.1 UART CLI commands

The CLI supports:

- `help, status`
- `mode idle|mod|cfg`
- `kp <float>, ki <float>, ref <float>`
- `bonus1 on|off, freq <float>, amp <float>`
- `bonus2 on|off`

Additionally, when the timed semaphore is active (after a button action), UART commands that modify parameters (`kp, ki, ref, bonus1, bonus2, freq, amp`) return `ERR uart locked` for 5 seconds. Monitoring and mode commands remain functional.

6.2 Buttons and on-board control

- MODE: cycles modes (PA0 polled, or PC13 EXTI).
- SELECT: toggles KP/KI in CONFIG.
- INC/DEC:
 - MOD: changes reference in steps of ± 0.05 (disabled when Bonus 1 is active).
 - CONFIG: changes KP or KI depending on selection.

7 Bonus Features

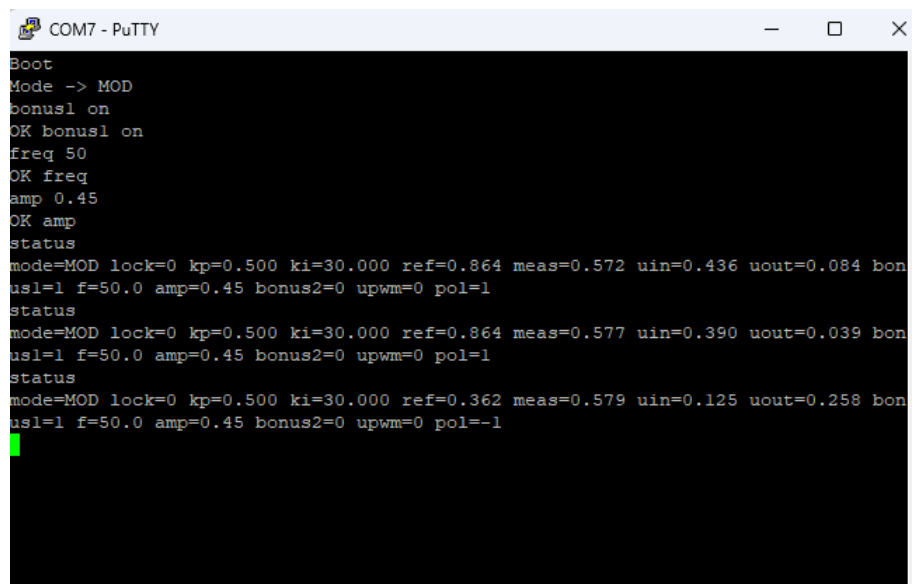
7.1 Bonus 1: inverter sinusoidal reference

When Bonus 1 is enabled, the reference is generated as:

$$\theta[k+1] = \theta[k] + 2\pi f Ts \quad (12)$$

$$r[k] = \text{clamp}(0.5 + a \sin(\theta[k]), 0, 1) \quad (13)$$

where frequency f is limited to $0.1 \leq f \leq 200$ and amplitude a is limited to $0 \leq a \leq 0.49$. Figure 2 shows the time-varying sinusoidal reference generated in inverter mode.



```
COM7 - PuTTY
Boot
Mode -> MOD
bonus1 on
OK bonus1 on
freq 50
OK freq
amp 0.45
OK amp
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.864 meas=0.572 uin=0.436 uout=0.084 bon
us1=1 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.864 meas=0.577 uin=0.390 uout=0.039 bon
us1=1 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.362 meas=0.579 uin=0.125 uout=0.258 bon
us1=1 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=-1
```

Figure 2. Bonus 1 enabled, the reference signal follows a sinusoidal trajectory mapped to the normalized range.

7.2 Bonus 2: PWM-event-driven plant stepping

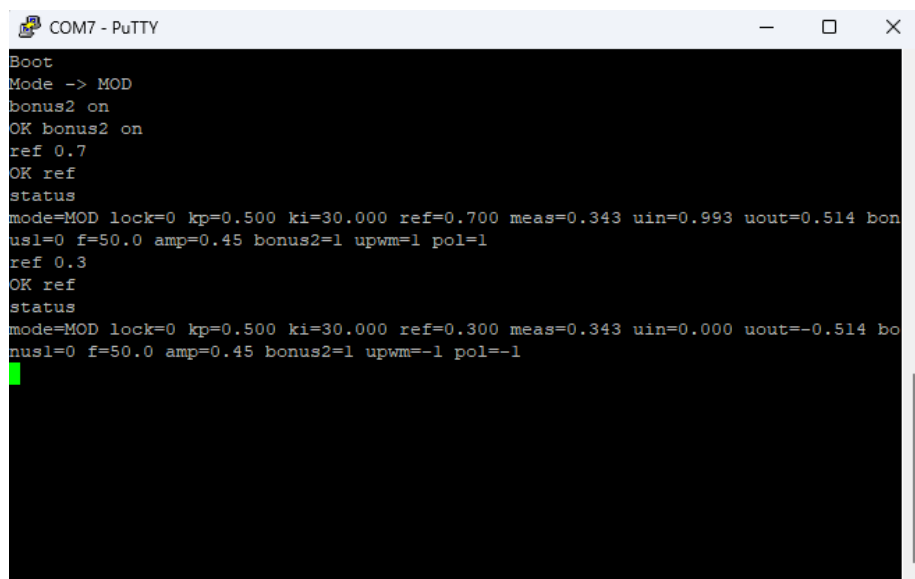
When Bonus 2 is enabled, the plant is stepped using PWM interrupt events:

- **PWM period start (TIM2 update):** apply input

$$u_{pwm} = \begin{cases} +1 & r[k] \geq 0,5 \\ -1 & r[k] < 0,5 \end{cases} \quad (14)$$

- **PWM pulse end (TIM2 compare):** apply input $u_{pwm} = 0$ during off-time.

This produces a switching-like input waveform $\{+1, 0, -1\}$ rather than an averaged duty input. As illustrated in Figure 3, the plant input is stepped using PWM timer events rather than an averaged duty input.



```
COM7 - PuTTY
Boot
Mode -> MOD
bonus2 on
OK bonus2 on
ref 0.7
OK ref
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.700 meas=0.343 uin=0.993 uout=0.514 bon
us1=0 f=50.0 amp=0.45 bonus2=1 upwm=1 pol=1
ref 0.3
OK ref
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.300 meas=0.343 uin=0.000 uout=-0.514 bo
nus1=0 f=50.0 amp=0.45 bonus2=1 upwm=-1 pol=-1
```

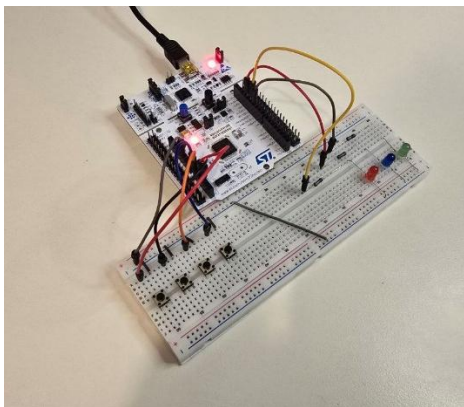
Figure 3. Bonus 2 enabled. Plant input is driven by PWM events with discrete input states and polarity switching.

8 Verification and Results

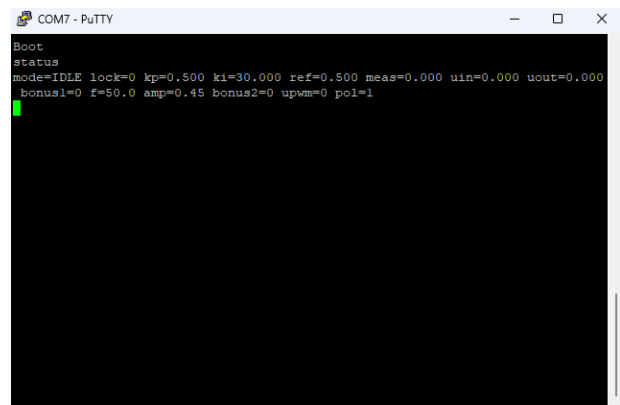
The system was verified experimentally using UART status outputs and on-board LED indicators. The verification focuses on correct mode transitions, closed-loop control behavior, parameter tuning, and the operation of the implemented bonus features. Each test case is supported by the corresponding figures.

8.1 Test 1: Boot and IDLE state

After reset, the system enters IDLE mode automatically. In this mode, the PWM duty cycle is set to zero and no control action is applied. UART status output confirms that the system is in IDLE mode with zero input and measurement values. Both mode indicator LEDs are off, indicating a safe operating state, as shown in Figure 4.



Board LEDs (IDLE mode)

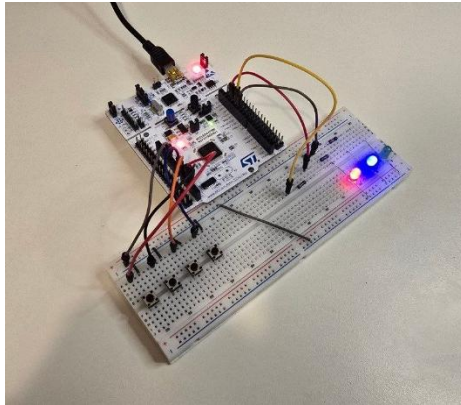


UART terminal output

Figure 4. LEDs are off and UART status confirms a safe startup state with zero PWM duty

8.2 Test 2: MOD mode tracking

In MOD mode, the PI controller regulates the plant output toward the selected reference. UART status outputs show the measured value converging toward the reference while a non-zero PWM duty cycle is applied. The PA6 LED is illuminated, confirming MOD mode operation. This behavior is illustrated in Figure 5.



Board LEDs (MOD mode)

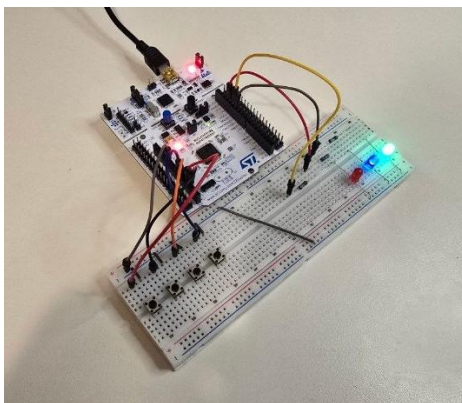
```
COM7 - PuTTY
Boot
status
mode=IDLE lock=0 kp=0.500 ki=30.000 ref=0.500 meas=0.000 uin=0.000 uout=0.000
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
Mode -> MOD
ref 0.3
OK ref
status
mode=MOD lock=0 kp=0.500 ki=30.000 ref=0.300 meas=0.271 uin=0.420 uout=0.573
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=-1
```

UART terminal output

Figure 5. MOD mode operation. PA6 LED indicates modulation mode and UART output shows closed-loop tracking of the reference.

8.3 Test 3: CONFIG mode tuning

CONFIG mode enables tuning of the controller gains using on-board buttons. A constant PWM duty cycle is applied to excite the plant, and changes to the proportional and integral gains are immediately visible in the UART status output. UART-based configuration is locked during this mode, indicated by the configuration lock flag. The PA7 LED confirms CONFIG mode operation, as shown in Figure 6.



Board LEDs (CONFIG mode)

```
COM7 - PuTTY
mode cfg
Mode -> CONFIG
status
mode=CONFIG lock=1 kp=0.500 ki=30.000 ref=0.500 meas=0.067 uin=0.100 uout=0.100
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
Select -> KI
KI increase
KI increase
status
mode=CONFIG lock=1 kp=0.500 ki=35.000 ref=0.500 meas=0.067 uin=0.100 uout=0.100
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
Select -> KP
KP decrease
KP decrease
status
mode=CONFIG lock=1 kp=0.400 ki=35.000 ref=0.500 meas=0.067 uin=0.100 uout=0.100
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
```

UART terminal output

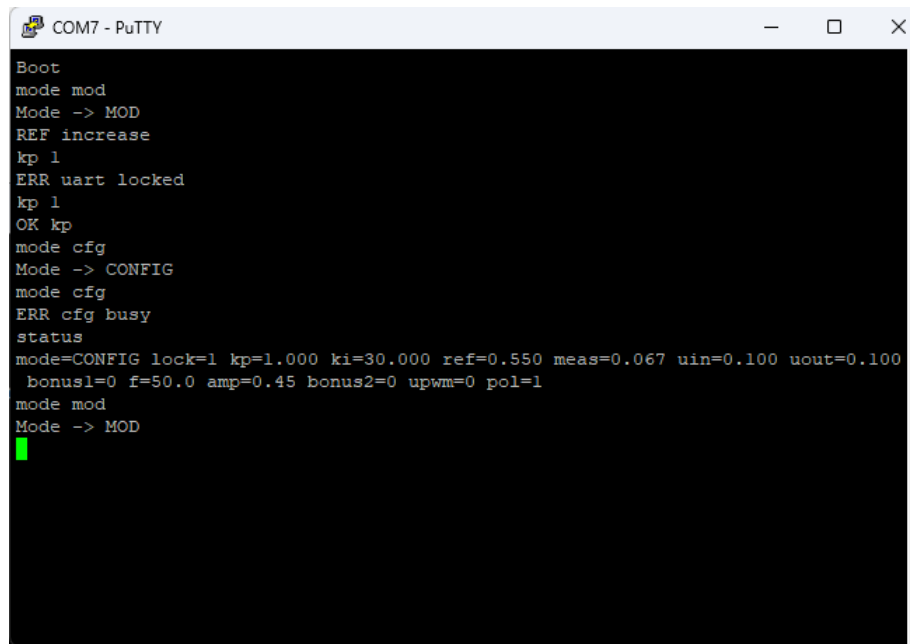
Figure 6. CONFIG mode. Parameter tuning is enabled, indicated by the PA7 LED and the UART configuration lock flag.

8.3.1 Interface arbitration and data protection verification

To verify correct data protection and user-interface arbitration, the following checks were performed:

- Entering CONFIG mode: CONFIG can be entered by button or UART. PA7 turns ON in both cases. When CONFIG is entered via UART, UART takes CONFIG ownership (binary semaphore), proven by a second `mode cfg` returning `ERR cfg busy`. The status output shows `lock=1` while in CONFIG.
- Timed UART blocking after button interaction: After any button interaction (MODE / INC / DEC / SELECT), UART commands that modify parameters (e.g. `kp`, `ki`, `ref`) are temporarily rejected for 5 seconds. During this period, the UART responds with `ERR uart locked`, while monitoring commands (`status`, `help`) remain available.
- Leaving CONFIG mode: Exiting CONFIG returns the system to normal operation. If CONFIG was entered via UART, leaving via `mode mod` releases the UART ownership semaphore, enabling normal button/UART interaction again.
- UART-owned CONFIG blocks buttons: When CONFIG is entered via UART (binary semaphore taken), button actions are ignored until UART leaves CONFIG.
- Exclusive CONFIG access enforcement: When CONFIG mode is entered via the UART, attempting to enter CONFIG mode again via the UART results in `ERR cfg busy`, demonstrating that the binary semaphore prevents multiple concurrent UART CONFIG entries.

These tests confirm that shared parameters are protected against concurrent access from the UART and button interfaces, ensuring deterministic and safe system behavior. Figure 7 shows the UART command-line output corresponding to this test, including the `ERR uart locked` response after button interaction and the `ERR cfg busy` response when attempting to re-enter CONFIG mode.



```
COM7 - PuTTY
Boot
mode mod
Mode -> MOD
REF increase
kp 1
ERR uart locked
kp 1
OK kp
mode cfg
Mode -> CONFIG
mode cfg
ERR cfg busy
status
mode=CONFIG lock=1 kp=1.000 ki=30.000 ref=0.550 meas=0.067 uin=0.100 uout=0.100
bonus1=0 f=50.0 amp=0.45 bonus2=0 upwm=0 pol=1
mode mod
Mode -> MOD
█
```

Figure 7: Verification of interface arbitration using binary and timed semaphores

8.4 Test 4: Bonus 1 sine reference

When Bonus 1 is enabled in MOD mode, the reference signal varies sinusoidally over time. Successive UART status outputs show the reference value changing continuously, confirming correct operation of the inverter-style reference generator. This behavior is shown in Figure 2.

8.5 Test 5: Bonus 2 PWM stepping

With Bonus 2 enabled, the plant input is driven by PWM timer events rather than by an averaged duty value. UART status outputs show discrete changes in input polarity and PWM state depending on the reference value. This confirms correct operation of the PWM-event-driven plant stepping, as illustrated in Figure 3.

9 Conclusion

This project successfully implements a real-time switched-mode power converter simulator on an STM32F4-series development board using the STM32 HAL. A sixth-order discrete-time state-space plant model and a PI controller are executed at a fixed sampling rate of 1 kHz, with PWM-based actuation and deterministic interrupt-driven timing.

Three operating modes [IDLE, MOD, and CONFIG] provide safe operation, closed-loop control, and systematic controller tuning. A UART command-line interface and on-board buttons enable flexible and robust user interaction, while software debouncing and configuration locking ensure reliable operation. The implementation demonstrates clear separation between hardware initialization and application logic, improving readability and maintainability.

Two bonus features extend the system beyond basic control. The inverter-style sinusoidal reference generator enables tracking of time-varying references, while PWM-event-driven plant stepping demonstrates event-synchronized simulation that more closely resembles switched-mode converter behavior. Experimental verification using UART status outputs and LED indicators confirms correct mode transitions, stable closed-loop behavior, and consistent user interaction.

Overall, the project meets all specified requirements and demonstrates a reliable real-time embedded control implementation suitable for further extension toward more advanced power electronic system simulations.

Appendix 1. Key Parameters

Table 3. Key Parameters

Parameters	Value
Sampling time T_s	1 ms (TS_SEC = 0.001)
TIM3 tick	1 kHz (Prescaler = 83, Period = 999)
TIM2 PWM	Prescaler = 83, Period = 999, Channel 1
Y_{FS}	Y_FULL_SCALE = 1.5
State limit	STATE_LIMIT = 50
Output limit	Y_LIMIT = 15
UART	USART2 @ 115200 baud, 8N1